

Wireless ADB Debugging Without Android Studio

<https://gemini.google.com/app/cdc16f03852d3ea2>

User prompt: i have android studio, i use it only to connect my andoird device/phone via wireless debug mode, to test my application, that running using npm. I feel andoird studio is dependancy, how to do the same without starting or using android studio

Response: You do not need Android Studio running to pair and connect your Android device over Wi-Fi. You only need the standalone Android SDK Platform-Tools (adb), which you already have installed.

1. Ensure ADB is in your PATH

The adb executable is located in your Android SDK folder:

- **macOS / Linux:** ~/Library/Android/sdk/platform-tools (macOS) or ~/Android/Sdk/platform-tools (Linux)
- **Windows:** %LOCALAPPDATA%\Android\Sdk\platform-tools

Open your terminal or command prompt and run:

```
adb version
```

Verification: If it outputs Android Debug Bridge version x.x.x, your PATH is set. If not, add the platform-tools directory to your system environment variables.

2. Method 1: Wi-Fi Pairing (Android 11+)

Ensure your phone and computer are on the same Wi-Fi network.

1. On your phone, go to **Settings > Developer options**.
2. Enable **Wireless debugging**, then tap directly on **Wireless debugging** to enter its settings screen.
3. Tap **Pair device with pairing code**. You will see an **IP address & Port** (e.g., 192.168.1.10:41234) and a 6-digit **Wi-Fi pairing code**.
4. In your terminal, run:

```
adb pair <IP_ADDRESS>:<PORT>
```

(Example: `adb pair 192.168.1.10:41234`)
5. Enter the 6-digit code when prompted. *Verification:* Terminal should display Successfully paired to
6. Return to the main **Wireless debugging** screen on your phone. Look at the **IP address & Port** shown directly under "IP address & Port" (note: the port here is different from the pairing port).
7. Connect to that port:

```
adb connect <IP_ADDRESS>:<PORT>
```

Verification: Run `adb devices`. You should see `<IP_ADDRESS>:<PORT>` device.

(Note: Pairing is a one-time step per Wi-Fi network. Next time, you only need to run `adb connect <IP_ADDRESS>:<PORT>`.)

3. Method 2: Quick Connect (Any Android Version)

If you have a USB cable nearby for the initial setup, you can avoid pairing codes entirely:

1. Connect your phone via USB cable with **USB Debugging** enabled.
2. In your terminal, set the ADB TCP port:

```
adb tcpip 5555
```

Verification: Terminal returns `restarting in TCP mode port: 5555`.

3. Unplug the USB cable.
4. Find your phone's IP address (in **Settings > About phone > Status information > IP address**).
5. Connect over Wi-Fi:

```
adb connect <PHONE_IP_ADDRESS>:5555
```

Verification: Run `adb devices`. Your phone will show up as `<PHONE_IP_ADDRESS>:5555` device.

Once connected via either method, your React Native / npm workflow (`npx react-native run-android`, `npm start`, etc.) will target the wireless device automatically.

User prompt: the studio showing me the connected devices, and able to mirror the mobile screen in the studio, will it be possible?

Response: Yes, you can do both device listing and full screen mirroring without touching Android Studio. In fact, Android Studio uses the standard Android debug tools under the hood for both features.

1. View Connected Devices

To see which devices are currently connected (USB or Wi-Fi):

```
adb devices
```

Verification: It will print a list like:

```
List of devices attached
192.168.1.15:5555    device
```

If you want to view phone details (model, manufacturer, Android version):

```
adb shell getprop ro.product.model
```

2. Mirror and Control Your Mobile Screen (Using **scrcpy**)

The industry standard for mirroring and controlling an Android screen from a PC is **scrcpy** (by Genymobile). It is free, open-source, ultra-lightweight, and significantly faster with lower latency than Android Studio's built-in mirror.

Installation:

- **Windows:** Run in PowerShell or Command Prompt:

```
winget install Genymobile.scrcpy
```

*(Or download the zip from the official GitHub releases and run **scrcpy.exe**)*

- **macOS:**

```
brew install scrcpy
```

- **Linux (Ubuntu/Debian):**

```
sudo apt install scrcpy
```

How to Mirror:

1. Connect your device wirelessly via `adb connect <IP>:<PORT>`.
2. Open your terminal and simply run:

```
scrcpy
```

Verification: A floating desktop window will immediately open displaying your real-time phone screen. You can interact with your phone using your computer's mouse and keyboard, drag-and-drop APKs to install them, and paste text directly.

Useful options for wireless mirroring:

- If your Wi-Fi is slow or stuttering, reduce the resolution and bitrate slightly:
- Keep the phone screen off while viewing it on your desktop (saves phone battery):

```
scrcpy -m 1024 -b 4M
```

```
scrcpy --turn-screen-off
```

3. Alternative: VS Code Extension

If you write code in **VS Code**, you can mirror your screen directly inside your editor:

1. Open VS Code Extensions (Ctrl+Shift+X or Cmd+Shift+X).
2. Search for and install **ADB GUI** or **Scrcpy**.
3. Once installed, it gives you a sidebar list of connected devices and a single click button to launch the mirrored screen right beside your code.

Sources

1. [Genymobile/scrcpy: Display and control your Android device - GitHub](#)
 2. [SCRCPY – Android Screen Mirroring Software \(Latest\)](#)
-

User prompt: i can see many Scrcpy versions, which one is trusted?

Response: The only official, trusted source for scrcpy is the **Genymobile/scrcpy** repository on GitHub, authored and maintained by Romain Vimont (@rom1v).

Third-party download portals and domain names imitating the tool (like unofficial .org or mirror sites) often repackage old versions or bundle bloatware.

Official & Safe Ways to Get It

1. Direct from the Official GitHub Releases (Portable ZIP)

- **Official Repository:** [GitHub – Genymobile/scrcpy](#)
- Go directly to the **Releases** section ([github.com/Genymobile/scrcpy/releases] (https://github.com/Genymobile/scrcpy/releases)).
- For Windows, download **scrcpy-win64-v<latest>.zip**. Extract it anywhere, and you will find **scrcpy.exe** and a bundled **adb.exe**.

2. Official Package Managers (Recommended) Package managers pull directly from verified manifests:

- **Windows (WinGet - built into Windows 10/11):**

```
winget install --exact Genymobile.scrcpy
```

- **macOS (Homebrew):**

```
brew install scrcpy
```

- **Linux (APT / Pacman / DNF):**

```
sudo apt install scrcpy
```

What About the "GUI" Versions?

Official **scrcpy** is strictly a command-line utility (running **scrcpy** in your terminal immediately pops up the mirrored window).

Third-party graphical wrappers exist (such as *guiscrcpy*, *QtScrcpy*, or *Escrcpy*) that add buttons and dropdown menus. You do not need any of them. Running **scrcpy** directly from PowerShell, Terminal, or a 1-line batch file is safer, faster, and avoids third-party tampering.

Sources

1. [Genymobile/scrcpy: Display and control your Android device - GitHub](#)
 2. [scrcpy/doc/windows.md at master - GitHub](#)
-

User prompt: i was talking about VS code

Response: Inside the VS Code Extensions Marketplace (Ctrl+Shift+X or Cmd+Shift+X), search for:

ADB Interface (Publisher: **barros**)

This is currently the most popular, well-maintained, and open-source option for managing devices and triggering screen mirrors.

What to Look For

When searching inside VS Code:

- **Publisher:** Verify the publisher name under the title (e.g., barros).
- **Open Source Repository:** Check the extension details panel to confirm it links to a public GitHub repo with active community support.
- **Avoid Bloated Forks:** Do not install extensions asking for separate license keys, external registrations, or bundled executables from unknown third-party domains.

How VS Code Extensions Use Scrcpy

Extensions in VS Code do not replace scrcpy; they are **GUI wrappers**. They rely on the official scrcpy and adb binaries already installed on your system to function:

1. Install the official scrcpy package via WinGet or Homebrew:

```
winget install --exact Genymobile.scrcpy
```

2. Install the **ADB Interface** extension in VS Code.

3. Open the **ADB** tab on the left sidebar:

- It detects devices found by `adb devices`.
- It provides buttons to **Connect over Wi-Fi** without memorizing port numbers.
- Clicking the **Mirror / Screen** icon runs the background scrcpy command and opens your phone display.

Built-in Alternative: VS Code Tasks (Zero Extension Dependency)

If you prefer not to trust any third-party VS Code extension, you can create a built-in VS Code task that launches scrcpy in one keystroke.

Create or update `.vscode/tasks.json` inside your project folder:

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "label": "Mirror Phone (scrcpy)",
```

```

        "type": "shell",
        "command": "scrcpy",
        "args": ["--always-on-top", "--turn-screen-off"],
        "problemMatcher": []
    }
]
}

```

Press **Ctrl+Shift+P** (or **Cmd+Shift+P**), select **Tasks: Run Task**, and choose **Mirror Phone (scrcpy)**. This uses only the official Genymobile tool with zero third-party plugin risk.

User prompt: on command scrcpy i got following: ERROR: Multiple (2) ADB devices over TCP/IP: ERROR: --> (tcpip) 192.168.1.4:43103 device moto_g_40__fusion ERROR: --> (tcpip) adb-ZD22245GF7-KRid7e._adb-tls-connect._tcp device moto_g_40__fusion ERROR: Select a device via -s (--serial), -d (--select-usb) or -e (--select-tcpip) ERROR: Server connection failed

Response: scrcpy failed because ADB registered your phone twice: once via the direct IP/port and once via mDNS/zero-config discovery (_adb-tls-connect._tcp). Because there are two active entries for the same device, scrcpy doesn't know which connection to target.

Quick Fix: Target the IP Directly

Specify the device identifier using the `-s` flag:

```
scrcpy -s 192.168.1.4:43103
```

Verification: The scrcpy window should open immediately and mirror your Moto G40 Fusion.

Permanent Fix: Clean Up Duplicate ADB Entries

To stop passing the `-s` flag every time, remove the duplicate connection:

1. Disconnect the mDNS entry:

```
adb disconnect adb-ZD22245GF7-KRid7e._adb-tls-connect._tcp
```

2. Verify only one device remains:

```
adb devices
```

Verification: You should only see 192.168.1.4:43103 device.

3. If both entries persist or keep colliding, restart the ADB daemon:

```
adb kill-server
adb connect 192.168.1.4:43103
scrcpy
```
